

Lenguaje Python

Cursada 2026

Archivos. Formatos JSON y CSV

Repaso 1: lambda

- ¿Qué hace el siguiente código?

```
In [ ]: # Nombre completo      Total Puntos Practica      Total TI      Total Pun

students = [['Bulbasaur', 79.25, 99.5, 24.25],
            ['Ivysaur', 95.0, 98.25, 62.5],
            ['Venusaur', 87.0, 91.75, 28.5],
            ['VenusaurMega Venusaur', 92.25, 98.0, 61.25],
            ['Charmander', 97.25, 91.25, 2.5],
            ['Charmeleon', 99.0, 91.75, 90.09],
            ['Charizard', 90.75, 95.75, 21.165],
            ['CharizardMega Charizard X', 85.0, 99.5, 92.5],
            ['Flareon', 96.5, 97.0, 92.915],
            ['Squirtle', 85, 98.25, 1.5]]

In [ ]: theory_approved = filter(lambda x: x[3] > 70, students)
        for student in theory_approved:
            print(f"{student[0]:.<30} {student[3]}")
```

Repaso 2: un poco más de lambda

- ¿Qué hace el siguiente código? ¿cuál es el tipo de las variables `is_working_age` y `correct_key_length`?

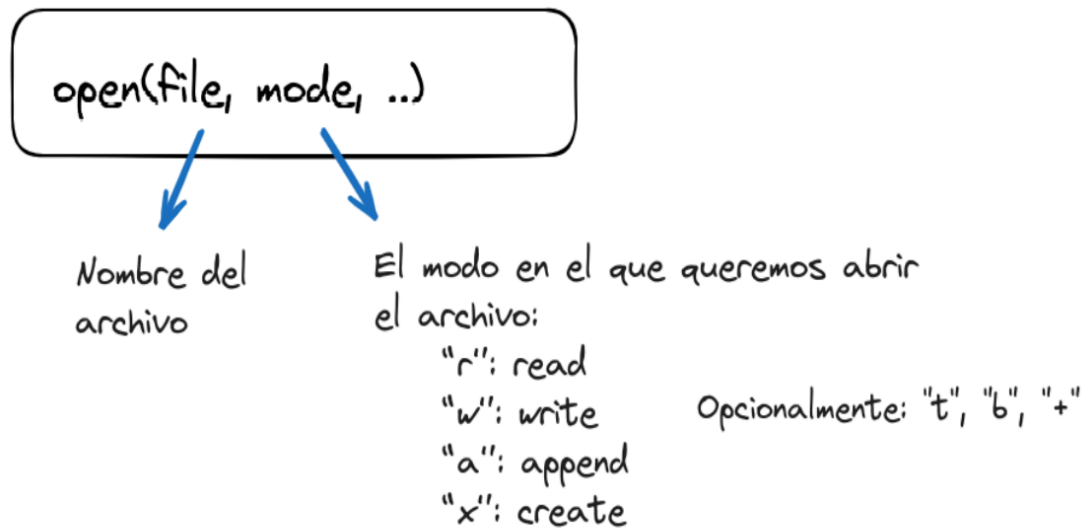
```
In [ ]: def validate(val_min, val_max):
        return lambda x: val_min <= x <= val_max

In [ ]: is_working_age = validate(18, 65)
        print(is_working_age(25))

In [ ]: correct_key_length = validate(8, 32)

        keyword = input('Escribí una clave:')
        print("La cantidad de caracteres es correcta." if correct_key_length(len(
```

Repaso 2: la función `open`



```
In [ ]: file = open('archivo.xxx')
```

Repaso 3: ¿qué pasa cuando el archivo está en otro directorio?

Usamos [pathlib](#) descrita en la [PEP 428](#): una interfaz orientada a objetos que permite el manejo de rutas.

```
In [ ]: from pathlib import Path
file = Path('ejemplos') / "clase05" / "calificaciones.csv"
file
```

¿Qué pasa si necesito guardar información que tiene una estructura?

Problema 1:

Tenemos que procesar los datos de los estudiantes de las tres comisiones de Python. Por cada estudiante contamos con su **DNI, nombre, comisión, puntos de teoría, puntos de práctica y ciudad de procedencia**.

Problema 2:

Queremos escribir un programa que nos permita gestionar mi música preferida. Para esta tenemos la siguiente información: **tema, intérprete y año**.

Problema 3:

Queremos gestionar el banco de preguntas para las autoevaluaciones realizadas en clase: **tema, enunciado, lista de opciones, respuestas correctas.**

En estos casos también podría usar un archivo de texto

¿Cómo se les ocurre? ¿Hicieron la tarea de la clase pasada?

Algunas posibilidades. Pensemos en el problema 2

Problema 2:

Queremos escribir un programa que nos permita gestionar mi música preferida. Para esto tenemos la siguiente información: **tema, intérprete y año.**

```
2015'      'Tema: Photograph - Intérprete: Ed Sheeran - Año:
---
          Tema: Photograph
          Intérprete: Ed Sheeran
          Año: 2015
          ---
          'Photograph-Ed Sheeran-2015'
          'Photograph**Ed Sheeran**2015'
```

- ¿Pros y contras?

Hay otras formas mejores...

JSON (JavaScript Object Notation)

- Es un **formato de intercambio de datos** muy popular. Por ejemplo:

```
{ "Tema": "Photograph",
  "Intérprete": "Ed Sheeran",
  "Año": 2015}
o
[{"Tema": "Unintended",
  "Intérprete": "Muse",
```

```
"Año": 1999},  
{"Tema": "Photograph",  
"Intérprete": "Ed Sheeran",  
"Año": 2015}]
```

- [+Info](#)

¿Sólo para archivos?

- Veamos este ejemplo: <https://sampleapis.com/api-list/movies>

Y éste es solo uno de los tantos ejemplos ...

Módulo json

- Python tiene un módulo que permite trabajar con este formato.
- Para usarlo, debemos importarlo.

```
In [ ]: import json
```

- Permite **serializar objetos**.

Proceso por el cual se convierte un objeto del programa Python (ejemplo una lista) en un formato de texto (en este caso: JSON).

Usando el módulo **json**:

- serializamos con: **dumps()** y **dump()**.
- deserializamos con: **loads()** y **load()**.

Más info en: <https://docs.python.org/3/library/json.html>

DESAFIO 1

Queremos guardar en un archivo información de la música que más me gusta.

Vamos a generar un archivo con la siguiente estructura:

- nombre de la banda o intérprete,
- ¿es_solista?
- ciudad y país en donde procede,
- una referencia a su trabajo.

Empecemos por los datos

¿Qué estructura de Python les parece mejor para almacenar los datos antes de querer guardarlos en un archivo?

```
In [ ]: data = [
    {"nombre": "William Campbell", "es_solista": False, "ciudad": "La Pla
    {"nombre": "Marcos Fava", "es_solista": True, "ciudad": "La Plata", "
    {"nombre": "Ed Sheeran", "es_solista": True, "ciudad": "Halifax", "pa
    {"nombre": "INXS", "es_solista": False, "ciudad": "Sidney", "pais": "
    {"nombre": "La Ley", "es_solista": False, "ciudad": "Santiago", "pais
    {"nombre": "Héroes del Silencio", "es_solista": False, "ciudad": "Zar
    {"nombre": "Panza", "es_solista": False, "ciudad": "Buenos Aires", "p
    {"nombre": "Adele", "es_solista": True, "ciudad": "Tottenham", "pais"
]
```

```
In [ ]: from pathlib import Path
file_route = Path('ejemplos') / "clase05" / "musica.json"
music_file = open(file_route, "w")
```

Ahora, vamos a guardar nuestros datos en el archivo:

Necesitamos **serializar** el objeto referenciado con **data**: usamos **json.dump**

```
In [ ]: import json

json.dump(data, music_file)
music_file.close()
```

Exploremos el archivo generado: [musica.json](#)

DESAFIO 2

Queremos visualizar los datos sobre mi música que previamente guardamos.

Ahora necesitamos **deserializar** para trabajar con nuestra lista de diccionarios: usamos **json.load**

```
In [ ]: #Recordemos que:
#file_route = Path('ejemplos') / "clase05" / "musica.json"

file = open(file_route, "r")
data = json.load(file)
print(data)

file.close()
```

¿De qué tipo de **data**?

```
In [ ]: type(data)
```

Abramos el [archivo](#) y observemos el dato booleano: ¿algo raro?

- Representación en Python
 - En Python, los valores booleanos se representan con **True** y **False** (con la primera letra en mayúsculas)
- Representación en JSON
 - En JSON, los valores booleanos se representan con **true** y **false** (en minúsculas)

Usa las siguientes tablas de conversión:

- [json-to-Python](#)
- [Python-to-json](#)

Un poco más legible

```
In [ ]: pretty_data = json.dumps(data, indent=4)
```

¿De qué tipo es **pretty_data**?

```
In [ ]: type(pretty_data)
```

```
In [ ]: print(pretty_data[:350])
```

En resumen - JSON

El **módulo json** permite acceder a las siguientes funciones:

- **json.dump()**: serializa un objeto Python a un objeto en formato JSON que puede almacenarse en un archivo.
- **json.load()**: deserializa un objeto JSON almacenado en un archivo a un objeto Python.

```
In [ ]: file_json = open("tempo.json", "w")
data = [{'first_name': 'Hilda', 'last_name': 'Lizarazu'}, {'first_name':
json.dump(data, file_json)
file_json.close()
```

```
In [ ]: file_json = open("tempo.json")
data = json.load(file_json)
data
```

En resumen - JSON (cont.)

El **módulo json** permite acceder a las siguientes funciones:

- **json.dumps():** convierte un objeto Python a un objeto **str**.
- **json.loads():** convierte el texto en formato JSON (puede ser str o byte) a un objeto Python.

```
In [ ]: data = [{'first_name': 'Hilda', 'last_name': 'Lizarazu'}, {'first_name':  
data_str = json.dumps(data)  
data_str
```

```
In [ ]: data = json.loads(data_str)  
data
```

CSV: ¿más formatos?

- **CSV (Comma Separated Values).**
- Es un formato muy común para importar/exportar desde/hacia hojas de cálculo y bases de datos.
- Ejemplo:

```
nombre,es_solista,ciudad,pais,referencia  
William Campbell,False,La  
Plata,Argentina,www.instagram.com/williamcampbellok  
Marcos Fava,True,La  
Plata,Argentina,https://open.spotify.com/intl-  
es/artist/2risap4rX5qNM54Gdiiuf1  
Ed Sheeran,True,Halifax,Reino  
Unido,https://open.spotify.com/intl-  
es/artist/6eUKZXXaKkcviH0Ku9w2n3V
```

- [+Info - PEP 305](#)

Datasets

Antes que nada: ¿qué es un dataset?

- Hay muchos datasets disponibles de muchas temáticas:
- Datos de gestión de gobiernos:
 - Har un portal de datos de [Argentina](#), de [CABA](#), de [pcia. de Buenos Aires](#).
 - Datos del [Banco mundial](#)
 - Portal [Kaggle](#)
 - [IMDB](#)
 - Y muchos más...

- Muchos son datos abiertos, pero otros... no tanto...
- ¡PRESTAR ATENCIÓN a la licencias y requisitos para su uso!
- Muchos en formato **CSV** y otros en **JSON**

Nuestra música en Spotify

Vamos a trabajar con el archivo: [songs_normalize.csv](#)

- Tenemos una copia de la [descripción del dataset](#) en donde se explica a qué se refiere cada campo.

```
In [ ]: import csv
file_route = Path('ejemplos') / "clase05" / "songs_normalize.csv"
```

```
In [ ]: file = open(file_route, "r")
csv_reader = csv.reader(file, delimiter=',')
```

```
In [ ]: #encabezado = csvreader.__next__()
header = next(csv_reader)
print(header)
```

```
In [ ]: file.close()
```

El módulo csv

- Hay que importarlo.
- **csv.reader**: crea un objeto **iterador** que nos permite recorrer las líneas del archivo.
- ¿Por qué incluimos el parámetro **delimiter**? ¿[Dialectos](#)?

```
csv_reader = csv.reader(file, delimiter=',')
```

DESAFIO 3

Busquemos cuántos temas hay de Adele en el dataset de Spotify.

Comencemos nuevamente el proceso.

```
In [ ]: file = open(file_route, "r")
csv_reader = csv.reader(file, delimiter=',')

#header = csvreader.__next__()
header = next(csv_reader)
print(header)
```

```
In [ ]: for line in csv_reader:
        if line[0] == "Adele":
```



```
print(f"{line[1]:.<30} {line[17]}")
file.close()
```

- ¿Cómo se accede a cada dato?
- ¿De qué tipo son **header** y **line**?

```
In [ ]: type(header)
```

Otra solución ...

```
In [ ]: file = open(file_route, "r")
csv_reader = csv.reader(file, delimiter=',')

adele_songs = filter(lambda x: x[0] == "Adele", csv_reader)
for elem in adele_songs:
    print(f"{elem[1]:.<30} {elem[17]}")

file.close()
```

Otra forma de acceder: csv.DictReader

```
In [ ]: file = open(file_route, "r")
csv_reader = csv.DictReader(file, delimiter=',')

adele_songs = filter(lambda x: x["artist"] == "Adele", csv_reader)

for elem in adele_songs:
    print(f"{elem['song']:.<30} {elem['genre']}")

file.close()
```

Hagamos la siguiente modificación a nuestro código:

■ Agregar un título al listado: "Adele en Spotify".

```
In [ ]: file = open(file_route, "r")
csv_reader = csv.DictReader(file, delimiter=',')
```

```
In [ ]: adele_songs = filter(lambda x: x["artist"] == "Adele", csv_reader)
print("Adele en Spotify")
#print("-"*16)
for elem in adele_songs:
    print(f"{elem['song']:.<30} {elem['genre']}")
#file.close()
```

No nos gusta y queremos separar un poquito el título para que quede mejor. Modifiquemos y volvamos a ejecutar... ¿Qué observamos?

Recordemos que ...

`csv_reader` es un **iterador** por lo que cuando llegamos al final de la iteración ya no vuelve a comenzar.

En un ejemplo más adelante veremos otra forma de trabajar teniendo en cuenta esto.

Creamos un archivo csv con nuestra música

- **csv.writer:** retorna un objeto que convierte los datos con los que trabajamos en el programa en cadenas con el formato delimitadas con el separador correspondiente.

```
In [ ]: file_route_json = Path('ejemplos') / "clase05" / "musica.json"
file_route_csv = Path('ejemplos') / "clase05" / "musica.csv"
```

```
In [ ]: #import csv
#import json

file_json = open(file_route_json)
file_csv = open(file_route_csv, "w")
```

```
In [ ]: music = json.load(file_json)
```

¿Qué contiene **music**? ¿De qué tipo es?

```
In [ ]: type(music)
```

```
In [ ]: # Creamos un objeto que nos permitirá guardar los datos en formato CSV
writer = csv.writer(file_csv)
```

```
In [ ]: # Guardamos el encabezado con los nombres de las columnas
writer.writerow(["nombre", "es_solista", "ciudad", "pais", "referencia"])
# Guardamos los datos
for elem in music:
    writer.writerow( [elem["nombre"], elem["es_solista"], elem["ciudad"],

file_json.close()
file_csv.close()
```

Exploremos el archivo [musica.csv](#)

Ahora lo leemos desde Python

```
In [ ]: #Recordemos que
#file_route_csv = Path('ejemplos') / "clase05" / "musica.csv"
```

```

file_cvs = open(file_route_csv, "r")
csv_reader = csv.reader(file_cvs, delimiter=',')
for line in csv_reader:
    print(line)
file_csv.close()

```

¿Notan algo respecto a los datos? Observemos el campo booleano.

DESAFIO 4

Queremos calcular las notas finales de los estudiantes de Python. Los datos los tenes guardados en el archivo CSV: [calificaciones.csv](#)

Las notas se calculan realizando un promedio pesado (30-70) entre los puntos de teoría y la práctica, filtrando primero aquellos estudiantes que hayan tenido más de 70 puntos en la teoría y en la práctica.

Los datos corresponden a:

- nombre de estudiante
- Total de práctica (100 pts. máx)
- Total de teoría (100 pts. máx)

```

In [ ]: # Abrimos los archivos
route_grades = Path('ejemplos') / 'clase05' / 'calificaciones.csv'
route_approved = Path('ejemplos') / 'clase05' / 'calificaciones_FINAL.csv'

file_grades = open(route_grades, 'r')
file_approved = open(route_approved, 'w')

# Creamos el iterador para acceder al archivo CSV
csv_reader = csv.DictReader(file_grades, delimiter=',')

```

```

In [ ]: # Primero: filtramos los aprobados
approved = list(filter(lambda student: float(student['Total Puntos Practica']) > 70, csv_reader))
#approved

```

```

In [ ]: # Segundo: calculamos la nota final
for student in approved:
    final_grade = float(student['Total Puntos Practica']) * 0.7 + float(student['Total Puntos Teoria']) * 0.3
    final_grade = round(final_grade)
    if 93 <= final_grade <= 100:
        student['FINAL'] = 9
    elif final_grade >= 86:
        student['FINAL'] = 8
    elif final_grade >= 78:
        student['FINAL'] = 7
    elif final_grade >= 70:
        student['FINAL'] = 6
#approved

```

```

In [ ]: # Tercero: generamos el archivo de salida y cerramos ambos archivos
writer = csv.writer(file_approved)
writer.writerow(['Nombre', 'Pts practica', 'Pts teoria', 'Nota Final'])

```

```
for student in approved:
    writer.writerow([student['Nombre'], student['Total Puntos Practica']],
file_grades.close()
file_approved.close()
```

En resumen - CSV

El **módulo csv** permite acceder a las siguientes funciones:

- **csv.reader(csvfile):** retorna un objeto iterable **reader** para procesar las líneas del archivo csvfile.
- **csv.writer(csvfile):** retorna un objeto **writer** responsable de convertir los datos del usuario en cadenas delimitadas en archivo csvfile.

```
In [ ]: import csv
file_csv = open("tempo.csv", "w")

writer = csv.writer(file_csv)
writer.writerow(['first_name', 'last_name'])
writer.writerow(["Hilda", "Lizarazu"])
writer.writerow(["Fabiana", "Cantilo"])
file_csv.close()
```

```
In [ ]: file_csv = open("tempo.csv")
csv_reader = csv.reader(file_csv, delimiter=',')
for line in csv_reader:
    print(line)
```

En resumen - CSV (cont.)

El **módulo csv** permite acceder a las siguientes funciones:

- **csv.DictReader(csvfile):** retorna un objeto **reader** pero asigna la información de cada fila a un diccionario cuyas claves se proporcionan mediante el parámetro opcional fieldnames o extraídos de la primera fila.
- **csv.DictWriter(csvfile):** crea un objeto **writer** pero utilizando un diccionario para generar las fileas en el archivos csvfile.

```
In [ ]: file_csv = open("tempo.csv", "w")
fieldnames = ['first_name', 'last_name']
writer = csv.DictWriter(file_csv, fieldnames=fieldnames)
writer.writeheader()
writer.writerow({'first_name': 'Hilda', 'last_name': 'Lizarazu'})
writer.writerow({'first_name': 'Fabiana', 'last_name': 'Cantilo'})
file_csv.close()
```

```
In [ ]: file_csv = open("tempo.csv")
reader = csv.DictReader(file_csv)
print(list(reader))
for row in reader:
    print(row['first_name'], row['last_name'])
```

JSON vs. CSV

- ¿Con qué tipo de datos trabajan? (¿texto o binario?)
- ¿En qué casos usamos archivos en formato json?
- ¿En qué casos usamos archivos en formato csv?

La sentencia with

La **sentencia with** crea un objeto denominado **runtime context** o **contexto de tiempo de ejecución** que permite ejecutar un grupo de sentencias bajo el control de un **administrador de contexto** o **context manager**. ¿Qué es esto?

Un administrador de contexto permite asignar y liberar recursos cuando se desee.

El ejemplo típico se da en el acceso a archivos, ya que son **recursos externos** a nuestros programas que requieren una gestión adecuada.

```
In [ ]: with open(route_approved) as file_csv:
        csv_reader = csv.reader(file_csv, delimiter=',')
        header, data = next(csv_reader), list(csv_reader)

        print(header)
        for line in data[:5]:
            print(line)
```

- ¿Dónde se cierra el archivo?
- ¿Por qué les parece que trabajamos **data** como una lista?

CONSIGNA: tarea para el hogar

Dado el conjunto de datos de Spotify, queremos:

1- Guardar en otro archivo, en formato json, las canciones que tienen asignado más de un género.

2- Los cinco (5) artistas con más canciones en el dataset durante el año 2019.

```
In [ ]: # Solución
```

Exploremos el siguiente dataset de películas

```
In [ ]: file_route = Path("ejemplos") / "clase05" / "mymoviedb.csv"

with open(file_route, "r") as file_csv:
    csv_reader = csv.DictReader(file_csv)
    columns = csv_reader.fieldnames
```

```
In [ ]: columns
```

DESAFIO 5

Queremos ver qué películas tienen más de 9980 votos.

Deberíamos trabajar con la columna `Vote_Count`

```
In [ ]: with open(file_route, "r") as file_csv:
        data = list(csv.DictReader(file_csv))
        movies = list(filter(lambda movie: movie['Vote_Count'] > '9980', data))
```

```
In [ ]: for movie in movies:
        print(f"{movie['Title']:.<30}{movie['Vote_Count']:>5}")
```

- ¿Hay algún problema?

Probar en casa: convertir a int. **Spoiler:** da error. ¿Por qué?

DESAFIO 6

Queremos ver los idiomas de las películas.

```
In [ ]: #columns
```

```
In [ ]: languages = map(lambda movie: movie['Original_Language'], data)
print(list(languages))
```

¿Nos interesa que se repita?

```
In [ ]: languages = set(map(lambda movie: movie['Original_Language'], data))
```

```
In [ ]: print(languages)
```

Notamos que hay datos que no son correctos.

IMPORTANTE: la etapa de limpieza de datos

Los datos no siempre están listos para ser procesados.

Seguramente se necesita una etapa previa al análisis que requiere explorar y analizar si los valores son los esperados o están completos.

¿Qué problemas puede traer procesar datasets sin tener en cuenta estos aspectos?

Más adelante veremos otras formas de procesar estos datasets.

DESAFIO 7

Queremos descargar el poster de las Spiderman.

```
In [ ]: data[:1]
```

```
In [ ]: spiderman = data[:1][0]
poster_spiderman = spiderman['Poster_Url']
poster_spiderman
```

```
In [ ]: import requests
image_route = Path("ejemplos") / "clase05" / "poster_spiderman.jpg"

image = requests.get(poster_spiderman)
with open(image_route, 'wb') as f:
    f.write(image.content)
```

El módulo `request` permite realizar peticiones HTTP.

En este caso, estamos realizando un GET de un recurso ubicado en la URL dada:
<https://image.tmbd.org/t/p/original/1g0dhYtq4irTY1GPXvft6k4YLjm.jpg>

CONSIGNA: tarea para el hogar

Dados los módulos de juegos mostrados en alguna clase anterior, modificar el código para que:

1. Se puedan jugar cada juego en forma independiente o desde el módulo juegos.
2. Si se juega desde el menú del módulo juegos, guardar en un archivo el día, hora en que se jugó cada juego.

Para la parte 2, deben elegir cuál es la mejor estructura y tipo de archivo utilizar.

- Si quieren, subir el código modificado a su repositorio en GitHub y compartir el enlace a la cuenta @clauBanchoff Y @sofiamartin.

Seguimos la próxima ...